

מדריך הצגת הפרויקט למרצה

פרויקט RAG רפואת ילדים — בנייה, קוד, הבנה
מה להגיד · איך לסביר · מה להדגים · שאלות צפויות

מבנה ההצגה המומלץ — 20 דקות

מטרה	זמן	נושא	שלב
לתפוס עניין — 'למה בכלל RAG?'	2 דק'	פתיחה + הבעיה	1
תמונה שלמה לפני פרטים	3 דק'	ארכיטקטורה ויזואלית	2
הדגמת הבנה טכנית	4 דק'	הסבר הקוד — build_index	3
זרימה מקצה לקצה	3 דק'	הסבר הקוד — retrieval + generation	4
מספרים אמיתיים + ניתוח	4 דק'	תוצאות הערכה + ablation	5
עומק הבנה — חשיבה ביקורתית	2 דק'	לקחים ושיפורים	6
גמישות מחשבתית	2 דק'	שאלות	7

עצה כללית

פתח תמיד עם 'למה' לפני 'מה'. המרצה רוצה לראות שהבנת את הבעיה שהטכנולוגיה באה לפתור, לא רק שכתבת קוד שעובד.

1

פתיחה — הבעיה שפותרים

מה לפתוח איתו:

RAG — שמאפשרת לשאול שאלות קליניות על ספרי לימוד ברפואת ילדים ולקבל תשובות מושרשות במקור, עם ציטוט מדויק לעמוד.
□ הפרויקט שלי בונה מערכת Retrieval-Augmented Generation

אז תסביר מיד: למה לא מספיק GPT/Claude רגיל?

אבל ממציאה פרטים. RAG מאלץ כל תשובה לבוא ישירות מהטקסט שאוחזר — שורה ממש מהספר — ומצטט את העמוד המדויק.
את המקרים הספציפיים בספר AAP 2022 — שמעון, ילד עם פרכוס חומי, אמה עם נשימה מהירה. הוא ייתן תשובה שנשמעת נכון
□ מודל שפה כללי לא יודע

נקודת זמן חשובה

שאלה ל-Claude ללא RAG — הוא מהזה תשובה שנשמעת רפואית אבל לא מופיעה בשום ספר. זה מוכיח שהבנת מתי RAG נחוץ.
תגיד את המשפט הזה: 'הבדיקה הפשוטה היא לשאול את אותה

הצג את תרשים הארכיטקטורה ועבור על כל שלב בשורה:

קבצי PDF

שני ספרים — 1,096 עמודים לאחר דילוג עמודי תמונה (פחות מ-50 תווים).

מספרי עמודים, שלוש שורות ריקות: PDF מסיר עיוותי `clean_text()`. חולץ טקסט עמוד-עמוד PyMuPDF **טעינה + ניקוי** `bullet` בודד.

(400 תווים, חפיפה 50) ופסקה (100–700 תווים לפי גבול טבעי). סה"כ 10,107 + 5,250 chunks. **פיצול לחלקים** שתי אסטרטגיות במקביל: גודל-קבוע

למכפלת נקודות. chunks כל-10 CPU שניות על L2. מודל 384-ממד, מנומל — all-MiniLM-L6-v2 **הטמעה** הנרמול הופך קוסינוס

נתונים חיצוני. במקור ChromaDB — הוחלף בגלל תקלת Windows שמנעה persistence בין תהליכים. **אינדקס numpy** קבצי .json + .npy. אין מסד

Claude haiku-פרומפט ל `top-5 chunks` → `np.argmaxpartition` → `qvec.T @ embeddings = scores` **חיפוש + יצירה** [Source: ...] תשובה + ציטוט →

טעות נפוצה

אבל גילית תקלת Windows ועברת ל-numpy. זה בדיוק מה שמרצה רוצה לשמוע — שנתקלת בבעיה אמיתית וחשבת על פתרון. אל תגיד 'השתמשתי ב-ChromaDB'. השתמשת ב-ChromaDB

פתח את הקובץ `src/build_index.py` ועבור על 4 הפונקציות הראשיות:

```
load_all_documents()
```

לכ חתופ PDF, צירם, דומע-דומע טסקט צלחם `clean_text()`, הנוב `section_map` המ-TOC. תמישר ריזחמ `{doc_id, text, metadata}`. ב היצקנופ-PyMuPDF `utils.py`.

```
chunk_fixed_size(docs, size=400, overlap=50)
```

קרזנ — מיות `tail < 30`. `chunk[text[i : i+400]]` פיסומ בלש לכ. `step = size - overlap = 350`. ררגנ וולח

```
embed_chunks(model, chunks, batch=128)
```

ריזחמ. `normalize_embeddings=True` → `L2 norm` → `cosine == dot product`. `float32 array (N, 384)`.

```
save_index(chunks, embeddings, name)
```

תיונוציח תיוולת 0 — `np.load` מע האירק. צבוקל `chunks` `json.dump` + `embeddings.npy` `f{name}_embeddings.npy`.

לחשב קוסינוס פשוט עם @ ולא צריך לחלק בגודל הוקטורים. זה גם מה שמאפשר לאחסן ב-.npy פשוט במקום מסד נתונים מיוחד. **כשאתה מראה את `embed_chunks`, תגיד: 'שים לב ל-`normalize_embeddings=True` — זה הסיבה שאפשר**

הקוד שכדאי להכיר בעל פה:

```
vectors = model.encode(texts, batch_size=128, normalize_embeddings=True)
np.save('fixed_embeddings.npy', vectors.astype(np.float32))
```

פתח `src/retrieval.py` — הראה את פונקציית `retrieve()`:

```
qvec = model.encode([query], normalize_embeddings=True).astype(np.float32) scores = (embeddings @ qvec.T).flatten() # (N,) cosine similarity top_idx = np.argmax(scores, -k)[-k:] # 0(N), no full sort top_idx = top_idx[np.argsort(scores[top_idx])[:-1]]
```

הכל, רק מוצא את ה- k הגדולים. אחרי שמצאתי את ה- k , אני ממיין רק אותם — זה הרבה יותר מהיר מ- argsort על כל המערך. ☐ הקוד הזה עושה חיפוש קוסינוס על 10,107 וקטורים תוך פחות ממאת שנייה. np.argmax הוא $O(N)$ — לא ממיין את

פתח src/rag_system.py — הראה את answer():

```
chunks = retrieve(query, k=5, strategy='fixed') context = '\n'.join(c['text'] for c in chunks) prompt = f'Context:\n{context}\n\nQuestion: {query}' response = client.messages.create(model='claude-haiku-4-5-20251001', max_tokens=512, messages=[...])
```

ביותר ראשון. $\text{max_tokens}=512$ מספיק לתשובה קלינית עם ציטוטים ומונע יצירת פסקאות ארוכות שגולשות מה- context . ☐ שים לב ש- context בנוי מ- chunks ממוינים לפי ניקוד יורד — הרלוונטי

5

תוצאות ההערכה וה-Ablation

תציג את הטבלאות מהדוח ותסביר כל מספר:

איך להסביר בפה	ערך	מדד
'בכמעט 83% מהשאלות, הטקסט שמכיל את התשובה מופיע בחמשת ה-chunks הראשונים'	0.827	$\text{Hit}@5$
'כמעט כל chunk שאוחזר רלוונטי — מעט רעש. זה גבוה מהצפוי בגלל שהקורפוס ממוקד.'	0.881	$\text{Precision}@5$
'הזמן מחולק: ~0.1s אחזור numpy, יצירה ב-Claude API. צוואר הבקבוק הוא ה-LLM, מזהה ~0.1s.'	3.14s	Latency
'הפתעה — צפינו לזה להיות הכי קשה. כללי הפרומפט עבדו טוב לאישור היעדר.'	8/8 מושלם	שליה = 1.000
'5' כשלים — רובם מונחי עץ-ההחלטה של קלייגמן שפיצול גודל-קבוע חתר במקום לא נכון.'	19/24 — הכי חלש	עובדתי = 0.792

זה המקום להראות הבנה עמוקה — Ablation

מה זה אומר — תגיד בקול	$\text{Precision}@k$	$\text{Hit}@k$	ניסוי
'הבסיס לעשרים השאלות הראשונות.'	0.810	0.900	fixed k=5 (בסיס)
'אותו Hit — אבל Precision גבוה ב-6%. פסקה מייצרת chunks יותר נכונים.'	0.870	0.900	para k=5
'אחד מספרי hit עולה כי פחות רעש — אבל מפסידים Precision.'	0.850	0.850	fixed k=3
'הוא נקודת האיזון k=5 — לא משתפר — רק מוסיפים רעש Hit.'	0.787	0.900	fixed k=8

פסקה נותנת 6% יותר precision בלי שום עלות בזמן תגובה. $k=8$ לא עוזר כי הוא מוסיף chunks שגויים שמבלבלים את המודל. ☐ תסביר את ה-ablation כך: 'הרצתי 4 ניסויים כדי להבין מה באמת משפיע. המסקנה המפתיעה: אסטרטגיית הפיצול חשובה יותר מ- k .

6

לקחים — מה הייתי משפר

זה השלב שמפריד בין 'השתמשתי בספרייה' לבין 'הבנתי את החומר':

מוסיף Cross-encoder. בנפרד chunk-מהיר אבל פחות מדויק — הוא מקודד שאילתה ו-Bi-encoder מחדש Cross-encoder לשאלות זמניות ו-10 top לדירוג ms-marco-MiniLM רואה את שניהם יחד ומדויק הרבה יותר. הייתי לדירוג multi-hop-עלות ~0.5 שניות, משתלם.

ומבטיח שכל chunk מתחיל ומסתיים במשפט שלם. זה ישפר קטגוריית 'מספרי' שבה ספרות נחתכות. **חפיפה ברמת משפט** עכשיו החפיפה היא 50 תווים — קטע משפט שרירותי. הייתי מחשב גבולות משפט קודם

המצוטט — מה שנקרא faithfulness check. זה מאפשר הערכה אוטומטית על כל 52 השאלות. כשופט LLM מספיקה. הייתי משתמש ב-Claude Sonnet כדי לבדוק אם התשובה שנוצרה מוסמכת על-ידי הטקסט (LLM-as-Judge) הערכה ידנית על 10 שאלות לא

□ המשפט שסוגר את ההצגה

הטכני פשוט יחסית — האתגר האמיתי הוא לאבחן איפה הכשלים ולהחליט מה לשפר קודם. ניתוח ה-ablation הוא הכלי לזה. תסיים עם: 'הלקח הכי חשוב שלמדתי הוא שהצינור

שאלות צפויות מהמרצה — ותשובות

קרא את זה כמה פעמים לפני ההצגה — אלה השאלות שמפרידות בין ציון 90 ל-100.

הבנה קונצפטואלית

ש: מה ההבדל בין RAG ל-fine-tuning?

Fine-tuning (catastrophic forgetting) שוכח — הוא 'צובע' את המשקולות של המודל בנתונים חדשים אבל עלול להשכיח ידע קיים. RAG גם עדכני יותר: אפשר לשנות את המסמכים בלי לאמן מחדש RAG. לא נוגע במשקולות — הוא מוסיף הקשר חיצוני בזמן ריצה RAG. טוב לידע עובדתי שמשתנה RAG; טוב לסגנון ולמשימות קבועות Fine-tuning.

ש: למה bi-encoder ולא cross-encoder?

על chunks לאינדקס של 10 (document), הוא צריך מעבר קדימה לכל זוג (שאיטלה — $O(N)$ מדויק יותר אבל Cross-encoder מכפיל מטריצה (dim-מחשב את הטמעת השאיטלה פעם אחת (384 Bi-encoder CPU. זה 10,000 קריאות לכל שאלה — כ-30 שניות. אם נדרש reranking ל-cross-encoder, חזורה bi-encoder: בהינתן אינדקס מוכן. לכן $O(1)$ — ואז

ש: מדוע L2 normalisation?

מילים. נרמול הופך את המדד לזווית בלבד (כיוון סמנטי) ומאפשר השוואה הוגנת בין שבר של שתי מילים מעץ-ההחלטה לפסקה ארוכה. ללא נרמול, מכפלת נקודות תלויה בגודל הוקטור — משפטים ארוכים יקבלו ניקוד גבוה יותר רק בגלל שיש בהם יותר

ש: מה זה Hit@k לעומת Precision@k?

שאוחרו רלוונטיים k-ממוצע — כמה מה Precision@k. אחרת 0, top-k אחד נכון ב chunk בינארי לשאלה — 1 אם לפחות Hit@k: Precision תוך שמירה על Hit מודד דיוק (כמה רעש יש?). המטרה היא למקסם Precision@k. (האם מצאנו) recall מודד Hit@k. הוא נקודת האיזון $k=5$ — גבוה

תהליך ה-RAG

ש: מה קורה אם ה-chunk הנכון לא אוחר?

במקורות. כשל ידוע: לעיתים המודל עדיין מצטט את ה-chunk הכי קרוב כ-fallback שגוי — זה failure mode שצוין בניתוח הכשלים. המערכת תענה על סמך מה שאוחר. אם אף chunk רלוונטי לא נמצא, הפרומפט מורה ל-Claude להגיד 'המידע לא נמצא'

ש: למה chunk size 400 ולא 200 או 800?

≈ 80 טוקנים — 2-כ משפטים, הקשר שלם, ניקוד ייחודי. זה אומת בניסוי k: ablation שונה לא שינה הרבה, אבל אסטרטגיית הפיצול כן. ההטמעה מקבלת הקשר דל ורעש גדל. 800: גדול מדי — שני נושאים קליניים מתחברים ל-chunk אחד, האות הסמנטי מדולל. 400: 200: קטן מדי — fragment של משפט אחד.

ש: מה ההבדל בין dense retrieval ל-BM25?

משמעות Dense retrieval. לא יוחרז chunk-אם הממביל לא מופיע בטקסט, ה — (TF-IDF variant) מחפש לפי מילות מפתח BM25. טוב לשפה טבעית Dense. 'criteria for afebrile child' יוחרז גם עמוד שכותב 'febrile seizure discharge' — מחפש לפי (טוב לשמות מדעיים ייחודיים) שמות תרופות.

שאלות טכניות

ש: למה החלפת ChromaDB ב-numpy?

בתהליך הבנייה אבל לא נכתב לדיסק כראוי. כל תהליך Rust-אינדקס ה: HNSW יש תקלת ChromaDB 1.5.9 אבל לא בין תהליכים — sleep+verify עם workaround ניסיתי. InternalError קיבל path חדש שפתח את אותו Python נטען בזיכרון לחלוטין, ו-0 תלויות נוספות portable K vectors, מספיק ל-10 numpy. עבד בתהליך אחד

ש: np.argsort לעומת np.argpartition — מה ההבדל?

אני ממין רק את ה $O(N)$ -הגדולים ב k מוצא את np.argpartition. $O(N \log N)$ — האלמנטים $N=10,107$ ממין את כל np.argsort אבל זה פתרון נכון, K elements, בממשות: לא הבדל גדול ל-10 $O(k \log k)$ — שנמצא $k=5$ -ללא מיון מלא. לאחר מכן

conceptually.

ש: למה הטמעת השאילתה ב-retrieve() ולא ב-rag_system.py?

Separation of concerns: retrieval.py לא יודע כלום על orchestrator הוא rag_system.py. אחראי לכל מה שנוגע לאינדקס retrieval.py בלבד — LLM. בנפרד בלי retrieval() זה גם מאפשר לבדוק vektors. —

ש: מה זה semantic drift?

קרובה אבל לא זהה. דוגמה: שאלה על אוטיזם מדיה מקבלת תשובה על דלקת אוזן כרונית כי שניהם מופיעים באותו chunk גדול. כשה-chunks ארוכים מדי, תשומת הלב של המודל 'מתפשטת' על הטקסט כולו ועשויה לענות על שאלה

שאלות 'מה אם'

ש: מה אם הקורפוס גדל ל-millions?

Qdrant/Weaviate או FAISS (approximate NN, GPU) מעבר ל: million בזיכרון. ל vectors מספיק ל-100 numpy flat-file — רק ה retrieval code ה-ontology. לפי תחום רפואי — שאלת ילד חולה לא צריכה לחפש ב (shard) יש גם לפצל. (distributed) backend-לא משתנה.

ש: מה אם מסמך מתעדכן מדי יום?

re-chunk רק אותו מסמך, החלף את השורות הרלוונטיות ב-.py ו-.json. לנפחי גבוהים: מסד נתונים עם upsert-by-ID כמו Qdrant. גישת incremental indexing: כל מסמך יש לו hash. כשה-mash משתנה — re-embed +

ש: מה אם שואלים שאלה שלא קשורה לרפואת ילדים?

פרומפט המערכת אומר לענות 'לא נמצא' אם ה-context לא רלוונטי. שיפור: הוסף classifier בשלב הראשון שבודק אם השאלה בתחום. המערכת תחזיר את ה-5 chunks הכי קרובים מה-corpus — שאלה על בישול תקבל חלקים רפואיים לא קשורים.

גיליון עזר מהיר — מושגים שחייבים להגיד נכון

טעות נפוצה להימנע ממנה	ההגדרה הנכונה בפה	מושג
'זה חיפוש' — זה לא רק חיפוש, זה גם יצירה מושרשת.	הזנת הקשר חיצוני שאוחזר לתוך פרומפט של LLM, מה שמאלץ את ה-LLM להיות מושרשת במסמכים ספציפיים.	Augmented Prompting
'מדידת מרחק' — זה מדידת זווית (cosine), לא מרחק אוקלידי.	הצגת וקטורי של טקסט ב- R^{384} כך שמשפטים דומים סמנטית קרובים יותר.	Embedding cosine distance
'חלוקה לפסקאות' — זה אסטרטגיה שבוחרים, לא רק פסקאות.	החלוקה של טקסט למסמכים קטנים יותר, למשל באמצעות Chunking.	Chunking
לא לבלבל עם Hit — Precision מדד recall, לא דיוק.	מדד בינארי: האם לפחות chunk אחד נכון מופיע ב-k התוצאות הראשונות?	Hit@k
לא 'כמה מהתשובות נכונות' — זה מדד על ה-chunks, לא על התשובות.	הערכת אספקת chunks שאוחזרו הם רלוונטיים? ממוצע על כל השאלות.	Precision@k
לא לבלבל עם cross-encoder שרואה שאלתה+מסמך יחד.	מודל שמקודד שאלתה ומסמך בנפרד ל-וקטורים, מה שמאפשר Precise comparison.	Cross-encoder
לא לומר 'normalization' בלי להסביר למה — התשובה: כתימת התשובה.	הנורמליזציה של וקטורים: $\text{Cosine similarity} = \frac{\text{dot product}}{\ a\ \ b\ }$	L2 Normalization
לא 'בדיקות' — ablation הוא ניסוי שיטתי, לא debug.	ניסוי שמשנה פרמטר אחד בכל פעם כדי לבדוד את השפעתו על הביצוע.	Ablation Study

עשה ואל תעשה בהצגה

אל תעשה	עשה
תגיד 'הכל עבד חלק' — מרצה לא מאמין לזה	דבר על ה-ChromaDB bug — זה מראה שנתקלת בבעיה אמיתית
תגיד סתם 'ניסיתי כמה ערכים'	הסבר את ה-ablation כניסוי שיטתי עם השערה
תסתמך רק על PDF — זה לא הדגמת קוד	הראה קוד אמיתי בטרמינל — לא רק סלידים
תמציא תשובה על מה שלא יודע — מרצה מרגיש	אמור 'לא ידעתי' ואחר כך הסבר מה למדת
תגיד 'בחרתי X כי X עובד' — תמיד יש מה לאבד	ציין את ה-tradeoff בכל החלטה (למה k=5 ולא k=3)
תדלג על הדגמה חיה — זה הרגע הכי חזק	תדגים run אחד ח'י: python src/rag_system.py 'שאלה'

סקריפט הדגמה חיה — הרץ את זה לפני ההצגה

פקודה לפתוח בטרמינל בתיקיית הפרויקט:

```
# python -c "from src.rag_system import answer; import json; r = answer('What are the discharge criteria for febrile seizure?'); print(r['answer']); print('Sources:', r['sources'])"
```

הפעל את ההערכה המלאה (תוצאות שמורות ב-eval_run.log):

```
python eval/run_eval.py --strategy fixed --k 5
```

הפעל ablation:

```
python eval/run_eval.py --ablation
```

תרגיל לפני ההצגה

מה ה-score, ולמה התשובה מצטטת בדיוק אותם sources. אם יש שגיאה — זה לא הסוף, אמור 'שגיאה מעניינת, בוא נבין ביחד'.
הרץ שאלה אחת חיה וסביר כל שורת פלט: chunk_id, אומר,

בהצלחה! — הפרויקט חזק, המספרים אמיתיים, והקוד עובד. הכל שנשאר הוא להסביר אותו בביטחון.